

UNIVERSIDADE FEDERAL DE SANTA CATARINA

Segurança da Computação

CTF - Detetives Criptográficos

Análise de Segurança e Exploração

Aluno: Artur Luiz Rizzato Toru Soda

Matrícula: 22200349

Data: 26/05/2026

Professores: Profa. Thaís Bardini Idalino e
Prof. Jean Everson Martina

Florianópolis, SC – 2026

Contents

1	Introdução	3
1.1	Objetivo técnico e critério de sucesso	3
2	Passo 1: Decodificação do Secret.txt	3
2.1	Informações extraídas neste passo	3
2.2	Metodologia e Vetor de Ataque (Reprodutibilidade)	3
2.2.1	Análise Inicial	3
2.2.2	Comandos executados	4
2.2.3	Resultado observado	4
2.3	Mapeamento de Primitivas Criptográficas	4
3	Passo 2: Descriptografia do Arquivo IUVA	5
3.1	Informações extraídas neste passo	5
3.2	Metodologia e Vetor de Ataque (Reprodutibilidade)	5
3.2.1	Análise Inicial	5
3.2.2	Comandos executados	5
3.2.3	Resultado observado	5
3.3	Mapeamento de Primitivas Criptográficas	7
4	Passo 3: Análise do Executável Verificador	7
4.1	Informações extraídas neste passo	7
4.2	Metodologia e Vetor de Ataque (Reprodutibilidade)	7
4.2.1	Comandos Utilizados	7
4.2.2	Resultado observado e hipótese de vulnerabilidade	8
4.3	Mapeamento de Primitivas Criptográficas	8
5	Passo 4: Geração de Identidade Forjada	9
5.1	Informações extraídas neste passo	9
5.2	Metodologia e Vetor de Ataque (Reprodutibilidade)	9
5.2.1	Comandos Utilizados	9
5.2.2	Resultado observado	9
5.3	Mapeamento de Primitivas Criptográficas	11
6	Passo 5: Assinatura de Documento Forjado	11
6.1	Informações extraídas neste passo	11
6.2	Metodologia e Vetor de Ataque (Reprodutibilidade)	11
6.2.1	Comandos Utilizados	11
6.2.2	Resultado observado	12
6.3	Mapeamento de Primitivas Criptográficas	12

7	Passo 6: Manipulação do Trust Store	13
7.1	Informações extraídas neste passo	13
7.2	Metodologia e Vetor de Ataque (Reprodutibilidade)	13
7.2.1	Comandos Utilizados	13
7.2.2	Resultado observado	13
7.3	Mapeamento de Primitivas Criptográficas	14
8	Passo 7: Teste Final - Aprovação do Verificador	14
8.1	Informações extraídas neste passo	14
8.2	Metodologia e Vetor de Ataque (Reprodutibilidade)	14
8.2.1	Comandos Utilizados	14
8.2.2	Resultado observado	15
8.3	Mapeamento de Primitivas Criptográficas	15
9	Recomendações de Mitigação	16
10	Conclusão Técnica	16

1 Introdução

Este relatório foi escrito no formato de registro técnico sequencial, acompanhando a resolução do desafio passo a passo. Em cada etapa, foi documentado:

- o que foi possível extrair como informação útil naquele momento;
- a metodologia e o vetor de ataque com comandos reproduzíveis;
- o mapeamento das primitivas criptográficas envolvidas e a forma de contorno;
- as evidências visuais de comprometimento (capturas de terminal).

Com isso, a narrativa permanece fluida e cronológica, desde a inspeção inicial do `secret.txt` até a aprovação final do documento forjado pelo `verificador`.

1.1 Objetivo técnico e critério de sucesso

O objetivo operacional deste trabalho foi reproduzir, de forma técnica e auditável, um fluxo de comprometimento da confiança do sistema alvo sem quebra matemática das primitivas criptográficas principais. O critério de sucesso adotado foi a aprovação, pelo `verificador`, de um documento forjado e assinado com identidade não legítima, acompanhada de evidências de terminal e mapeamento das primitivas envolvidas.

2 Passo 1: Decodificação do Secret.txt

2.1 Informações extraídas neste passo

Da apresentação, sabemos que o `secret.txt` possui exatamente 64 caracteres ASCII (512 bits) e representa a pista inicial para as próximas etapas.

2.2 Metodologia e Vetor de Ataque (Reprodutibilidade)

2.2.1 Análise Inicial

O arquivo `secret.txt` contém:

```
1 Teve$ywev$$zivmjmgehsv$gsrwyxpi
2 $ew$mrxyv??iw0$oAwikvihs$EIWcGFG
```

Características:

- Tamanho: 64 caracteres ASCII (512 bits)
- Formato: Texto cifrado em duas linhas
- Tipo de cifra: Variação de Caesar Cipher

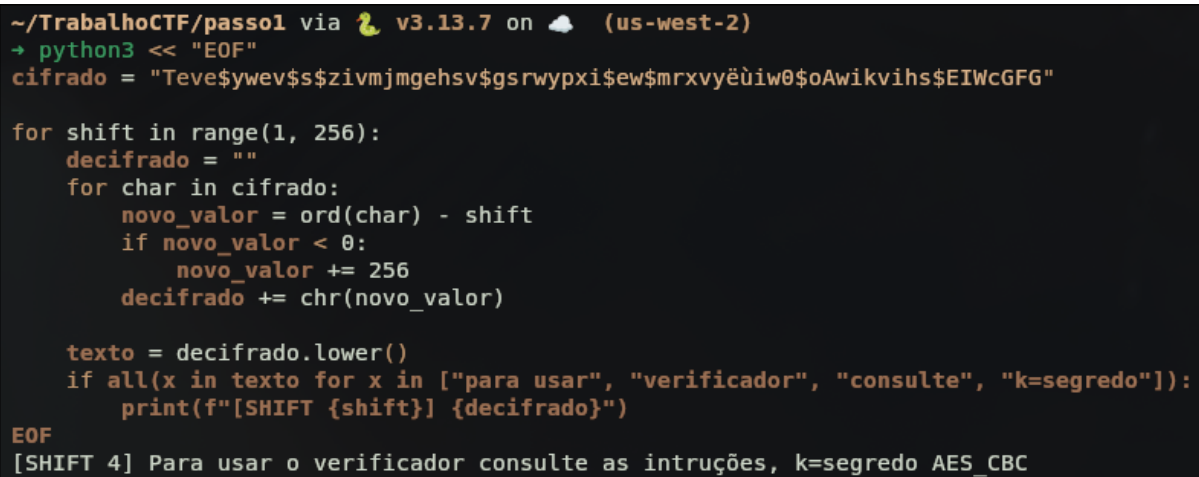
2.2.2 Comandos executados

Para quebrar a cifra, utilizamos força bruta testando todos os shifts possíveis (0-255):

```
1 cifrado = "Teve$ywev$$zivismjmghe$sv$gsrwyxpi$ew$mrxyv??  
    iw0$oAwikvihs$EIWcGFG"  
2  
3 for shift in range(1, 256):  
4     decifrado = ""  
5     for char in cifrado:  
6         novo_valor = ord(char) - shift  
7         if novo_valor < 0:  
8             novo_valor += 256  
9         decifrado += chr(novo_valor)  
10  
11     texto = decifrado.lower()  
12     if all(x in texto for x in ["para_usar", "verificador", "  
        consulte", "k=segredo"]):  
13         print(f"[SHIFT_{shift}]_{decifrado}")
```

Listing 1: Script Python para decodificar

2.2.3 Resultado observado



```
~/TrabalhoCTF/passol via 🐍 v3.13.7 on ☁️ (us-west-2)  
→ python3 << "EOF"  
cifrado = "Teve$ywev$$zivismjmghe$sv$gsrwyxpi$ew$mrxyvëüiw0$oAwikvihs$EIWcGFG"  
  
for shift in range(1, 256):  
    decifrado = ""  
    for char in cifrado:  
        novo_valor = ord(char) - shift  
        if novo_valor < 0:  
            novo_valor += 256  
        decifrado += chr(novo_valor)  
  
    texto = decifrado.lower()  
    if all(x in texto for x in ["para usar", "verificador", "consulte", "k=segredo"]):  
        print(f"[SHIFT {shift}] {decifrado}")  
EOF  
[SHIFT 4] Para usar o verificador consulte as instruções, k=segredo AES_CBC
```

Figure 1: Output do script mostrando a decodificação bem-sucedida

Chave extraída: **segredo**

2.3 Mapeamento de Primitivas Criptográficas

Neste passo, a primitiva principal foi uma cifra de substituição do tipo Caesar sobre bytes ASCII. O propósito arquitetural era esconder a senha do próximo estágio sem expor o

segredo em texto claro. O contorno foi feito por força bruta completa no espaço de deslocamentos (0–255), com validação por legibilidade do texto em português e identificação de padrão de segredo.

3 Passo 2: Descriptografia do Arquivo IUVA

3.1 Informações extraídas neste passo

A mensagem decodificada no Passo 1 indica a chave `segredo` e o uso de `AES_CBC`. Isso direciona diretamente a tentativa de decriptação do arquivo `IUVA`.

3.2 Metodologia e Vetor de Ataque (Reprodutibilidade)

3.2.1 Análise Inicial

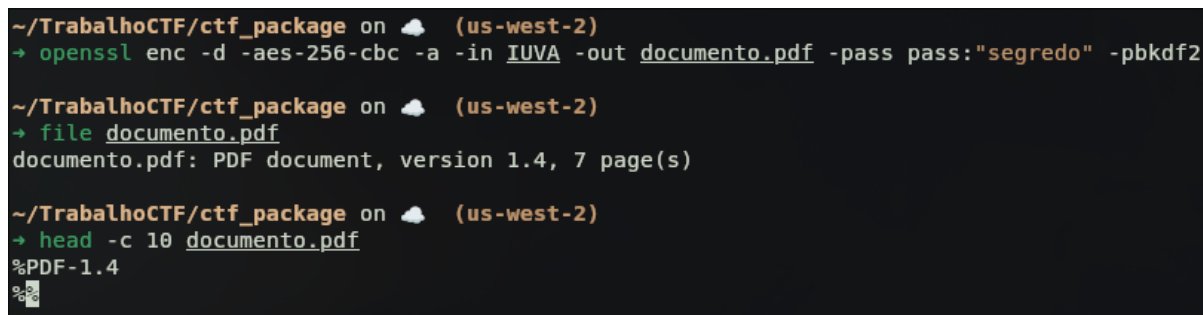
O arquivo `IUVA` é um binário criptografado originalmente em formato `PDF`, usando criptografia simétrica moderna (`OpenSSL`).

3.2.2 Comandos executados

```
1 openssl enc -d -aes-256-cbc -a -in IUVA -out documento.pdf \
2   -pass pass:"segredo" -pbkdf2
3
4 file documento.pdf
5
6 head -c 10 documento.pdf
```

Listing 2: Descriptografar com OpenSSL

3.2.3 Resultado observado



```
~/TrabalhoCTF/ctf_package on 🌩 (us-west-2)
→ openssl enc -d -aes-256-cbc -a -in IUVA -out documento.pdf -pass pass:"segredo" -pbkdf2

~/TrabalhoCTF/ctf_package on 🌩 (us-west-2)
→ file documento.pdf
documento.pdf: PDF document, version 1.4, 7 page(s)

~/TrabalhoCTF/ctf_package on 🌩 (us-west-2)
→ head -c 10 documento.pdf
%PDF-1.4
%❏
```

Figure 2: PDF descriptografado com sucesso - validação do header

Instruções de Uso do Verificador de Assinaturas (IUVA)

 **CLASSIFICAÇÃO: CONFIDENCIAL – USO INTERNO ESTRITO**

Versão do Documento: 2.2.0 (*Atualização de Segurança*)

Ferramenta Alvo: verificador

1. Visão Geral do Sistema

O **Verificador de Assinaturas** é a ferramenta central de validação de comunicação interna do nosso grupo. Para garantir a autenticidade, o não-repúdio e a integridade das ordens emitidas pelos líderes da operação, todas as mensagens devem passar por uma dupla camada de verificação (Simétrica e Assimétrica) antes de qualquer execução.

Este documento estabelece as diretrizes técnicas de funcionamento do **verificador** e as regras de gestão da nossa infraestrutura de chaves (PKI) descentralizada.

2. Arquitetura de Confiança e Verificação

Devido à necessidade de operarmos em ambientes isolados (*offline/air-gapped*), nosso verificador não se comunica com servidores externos ou Autoridades Certificadoras (CAs) globais.

Adotamos um modelo de **Confiança Baseada em Diretório Local**. A aprovação de uma assinatura depende exclusivamente de o certificado público do emissor estar previamente catalogado e autorizado na infraestrutura da máquina onde a ferramenta está sendo executada.

O Processo de Validação (v2.2.0)

Quando o **verificador** é executado, ele exige quatro componentes e realiza as seguintes etapas em milissegundos:

Passo	Camada de Segurança	Ação do Verificador

Figure 3: Página inicial do PDF descriptografado

3.3 Mapeamento de Primitivas Criptográficas

Neste passo, a primitiva utilizada pelo sistema foi criptografia simétrica AES-256-CBC via OpenSSL. O propósito arquitetural era preservar confidencialidade do documento IUVA. A proteção não foi quebrada criptograficamente; o contorno ocorreu com uso legítimo da chave obtida no Passo 1 para realizar a operação de decifração.

4 Passo 3: Análise do Executável Verificador

4.1 Informações extraídas neste passo

Com o conteúdo do PDF acessível, a próxima etapa é entender como o `verificador` decide se um documento e sua assinatura são confiáveis. Os testes iniciais também mostraram que a ferramenta exige quatro entradas no fluxo final: mensagem, assinatura, certificado e HMAC-SHA256.

4.2 Metodologia e Vetor de Ataque (Reprodutibilidade)

4.2.1 Comandos Utilizados

```
1 file verificador
2 chmod +x verificador
3
4 # Executar para ver ajuda
5 ./verificador
6
7 # Extrair strings interessantes
8 strings verificador | grep -i "usage\|verify\|sign\|key\|trust"
9
10 # Procurar por arquivos de chave/certificado
11 find . -name "*.pem" -o -name "*.key"
12
13 # Explorar trust store
14 ls -la .trust_store/
```

Listing 3: Exploracao inicial do verificador

4.2.2 Resultado observado e hipótese de vulnerabilidade

```
~/TrabalhoCTF/ctf_package on ☁ (us-west-2)
→ file verificador
verificador: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=d3d060968c2b8c41aa17bbc6e127f0d462c98025, stripped

~/TrabalhoCTF/ctf_package on ☁ (us-west-2)
→ chmod +x verificador

~/TrabalhoCTF/ctf_package on ☁ (us-west-2)
→ ./verificador
=====
VERIFICADOR DE ASSINATURAS E CONFIANÇA - v2.2.0
=====
Uso incorreto da ferramenta.
Sintaxe: ./verificador <mensagem.txt> <assinatura.bin> <certificado.pem> <hmac_sha256>

~/TrabalhoCTF/ctf_package on ☁ (us-west-2)
x strings verificador | grep -i "usage\|verify\|sign\|key\|trust"

signal
pyi-bootloader-ignore-signals
install signal handlers
Failed to set 'install signal handlers': %s
Failed to signal the sync semaphore!
3key
signal)

~/TrabalhoCTF/ctf_package on ☁ (us-west-2)
→ find . -name "*.pem" -o -name "*.key"

./trust_store/root_ca.pem
./verificador.pem

~/TrabalhoCTF/ctf_package on ☁ (us-west-2)
→ ls -la .trust_store/
total 12
drwxrwxr-x 2 artur artur 4096 May  7 16:30 .
drwxrwxr-x 3 artur artur 4096 May 12 18:24 ..
-rw-rw-r-- 1 artur artur 1107 May  7 15:43 root_ca.pem
```

Figure 4: Análise do verificador - saídas da etapa de reconhecimento

Hipótese de vulnerabilidade: o desenho de confiança parece estar concentrado no conteúdo local de `.trust_store`. Se o verificador aceitar qualquer certificado encadeado (ou tratado como confiável) a partir desse diretório, então a inserção de um certificado forjado permite contornar a autenticação sem quebrar RSA. Além disso, a presença explícita de `<hmac_sha256>` na sintaxe e strings internas relacionadas a segredo indicam que a proteção de integridade adicional depende de material embarcado no próprio binário.

Na fase de análise estática aprofundada, a constante de integridade utilizada pelo programa foi identificada como `1nt3gr1d4d3`, removendo a ambiguidade da pista inicial e permitindo reproduzir corretamente o cálculo do HMAC no Passo 7.

4.3 Mapeamento de Primitivas Criptográficas

Neste passo, não houve quebra direta de algoritmo criptográfico, e sim reconhecimento da arquitetura de confiança. A análise mostrou dependência de artefatos de assinatura digital (RSA/SHA-256), certificados X.509 e um valor HMAC-SHA256 consumidos pelo verificador. O contorno foi preparatório: identificar pontos de validação e os caminhos de trust store para, nos próximos passos, introduzir identidade forjada aceita pelo fluxo de verificação.

5 Passo 4: Geração de Identidade Forjada

5.1 Informações extraídas neste passo

A análise indica que, se conseguirmos introduzir uma identidade assinante dentro da confiança do sistema, o verificador tenderá a aceitar mensagens forjadas.

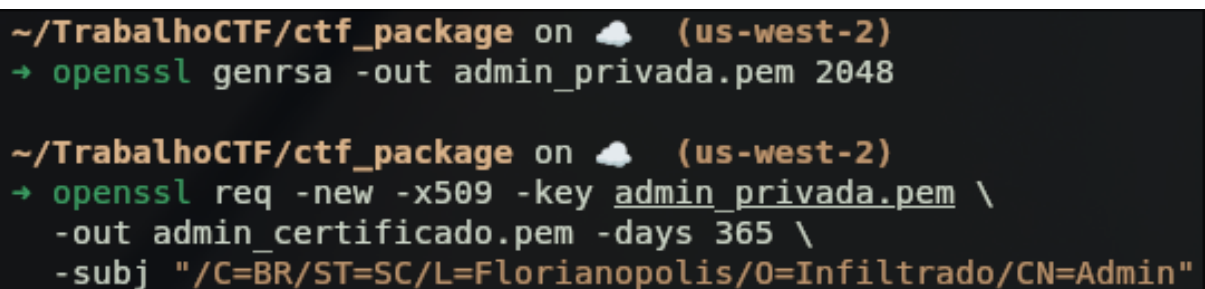
5.2 Metodologia e Vetor de Ataque (Reprodutibilidade)

5.2.1 Comandos Utilizados

```
1 # Gerar chave privada RSA
2 openssl genrsa -out admin_privada.pem 2048
3
4 # Gerar certificado autoassinado
5 openssl req -new -x509 -key admin_privada.pem \
6   -out admin_certificado.pem -days 365 \
7   -subj "/C=BR/ST=SC/L=Florianopolis/O=Infiltrado/CN=Admin"
8
9 # Verificar o certificado
10 openssl x509 -in admin_certificado.pem -text -noout
```

Listing 4: Gerar chave privada e certificado

5.2.2 Resultado observado



```
~/TrabalhoCTF/ctf_package on 🌩 (us-west-2)
→ openssl genrsa -out admin_privada.pem 2048

~/TrabalhoCTF/ctf_package on 🌩 (us-west-2)
→ openssl req -new -x509 -key admin_privada.pem \
  -out admin_certificado.pem -days 365 \
  -subj "/C=BR/ST=SC/L=Florianopolis/O=Infiltrado/CN=Admin"
```

Figure 5: Execução da geração de chave e certificado forjado

```

~/TrabalhoCTF/ctf_package on 🌩 (us-west-2)
→ openssl x509 -in admin_certificado.pem -text -noout
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      74:9a:74:4a:3a:d4:6d:07:49:0f:38:6a:a6:16:28:54:81:ca:e9:b7
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: C=BR, ST=SC, L=Florianopolis, O=Infiltrado, CN=Admin
    Validity
      Not Before: May 12 23:38:41 2026 GMT
      Not After : May 12 23:38:41 2027 GMT
    Subject: C=BR, ST=SC, L=Florianopolis, O=Infiltrado, CN=Admin
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      Public-Key: (2048 bit)
      Modulus:
        00:ea:f5:bc:a1:8e:a7:ac:51:ed:92:f9:29:d1:d5:
        30:70:a7:71:1f:a6:20:24:16:1b:47:ad:41:a4:54:
        dc:b9:f5:6d:69:3d:a4:26:4b:72:4d:59:6a:f4:1d:
        73:95:ee:7c:60:fd:81:69:b3:c8:ed:83:f0:97:4f:
        28:0a:aa:5d:6c:23:e6:2e:0f:d8:c1:0a:6e:45:e3:
        40:45:14:fd:3f:ef:ee:ce:db:9c:54:0d:64:96:fe:
        c5:a1:33:0b:71:9a:b1:97:17:d3:0b:26:9b:9d:e2:
        9e:c5:b1:3c:63:46:de:58:ae:54:40:1d:09:31:8c:
        f6:13:72:9d:62:67:ad:4d:44:8f:a6:0f:f5:ef:fa:
        a8:df:03:68:9a:31:2c:c6:7c:51:e3:31:3d:c1:84:
        04:d7:e8:98:f3:aa:03:15:f5:44:97:a1:1e:e0:f4:
        fe:a2:3d:dc:31:6a:4d:25:b8:ee:fc:65:40:38:72:
        a7:8e:e5:18:df:3d:b1:9e:08:e3:37:fe:0a:f5:13:
        53:98:f2:74:bf:8c:a7:b8:fd:61:3c:90:08:dd:1b:
        66:57:f0:fa:8b:5c:8a:b8:92:8f:46:8e:7d:68:68:
        e2:18:0d:cb:46:26:96:99:61:d0:89:fd:45:d1:d1:
        c2:1d:30:e4:19:41:cd:8f:5c:91:fe:6b:41:c1:29:
        d3:3b
      Exponent: 65537 (0x10001)
    X509v3 extensions:
      X509v3 Subject Key Identifier:
        2B:25:01:3C:6F:6F:29:23:70:8E:54:48:B7:B8:70:18:E5:C6:81:2F
      X509v3 Authority Key Identifier:
        2B:25:01:3C:6F:6F:29:23:70:8E:54:48:B7:B8:70:18:E5:C6:81:2F
      X509v3 Basic Constraints: critical
        CA:TRUE
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
      bc:ba:5e:56:9e:74:df:7e:2d:9d:67:45:17:93:23:11:ba:80:
      9c:52:82:c6:2f:bb:6f:9c:f0:57:e5:6e:d9:35:81:37:c9:18:
      c2:d6:b3:3c:b2:2a:76:51:22:b1:0b:6c:20:84:61:e6:49:0f:
      27:04:a3:db:5c:36:4e:c8:31:84:34:2e:27:61:bf:e3:ce:7a:
      d5:60:b9:5e:ca:71:c9:c5:4d:eb:90:9e:6c:4e:ca:ff:bc:50:
      48:ce:f3:61:10:d8:18:79:64:17:d7:95:b4:6f:84:78:04:da:
      6d:14:3a:a1:89:b5:74:74:60:65:d1:47:6e:30:58:ac:f6:29:
      29:71:42:72:42:34:b8:33:68:14:9b:a2:68:5d:2c:f7:10:23:
      a4:1c:3f:aa:46:5e:3f:dd:bf:76:1a:8e:a8:ab:6a:e1:d1:b6:
      67:d2:e9:da:62:6a:6e:92:26:12:ee:0a:68:40:f6:61:c8:16:
      06:4d:66:10:ff:b1:70:b2:c8:63:dc:7f:45:51:02:68:60:5e:
      b5:74:e2:1d:dd:e5:19:c3:d8:45:75:4b:b1:e6:fc:78:f4:83:
      b2:ce:13:16:f5:8b:c4:53:19:8b:b1:c7:dc:e3:68:21:23:c4:
      1b:2b:5f:8b:cc:88:a2:db:bd:72:cc:6b:26:31:c1:8e:4c:5c:
      6c:66:8a:41

```

Figure 6: Verificação do certificado forjado

Foram criados dois arquivos principais: **admin_privada.pem** (chave privada RSA-2048) e **admin_certificado.pem** (certificado X.509 autoassinado com atributos administrativos).

5.3 Mapeamento de Primitivas Criptográficas

As primitivas deste passo foram criptografia assimétrica RSA-2048 e certificado digital X.509. O propósito arquitetural dessas primitivas é vincular uma identidade a uma chave pública confiável. O contorno consistiu em gerar localmente um par de chaves e um certificado autoassinado com atributos administrativos, explorando a ausência de validação robusta da cadeia de certificação.

6 Passo 5: Assinatura de Documento Forjado

6.1 Informações extraídas neste passo

Com a identidade forjada criada, o próximo teste lógico é produzir um artefato assinado que possa ser submetido ao verificador.

6.2 Metodologia e Vetor de Ataque (Reprodutibilidade)

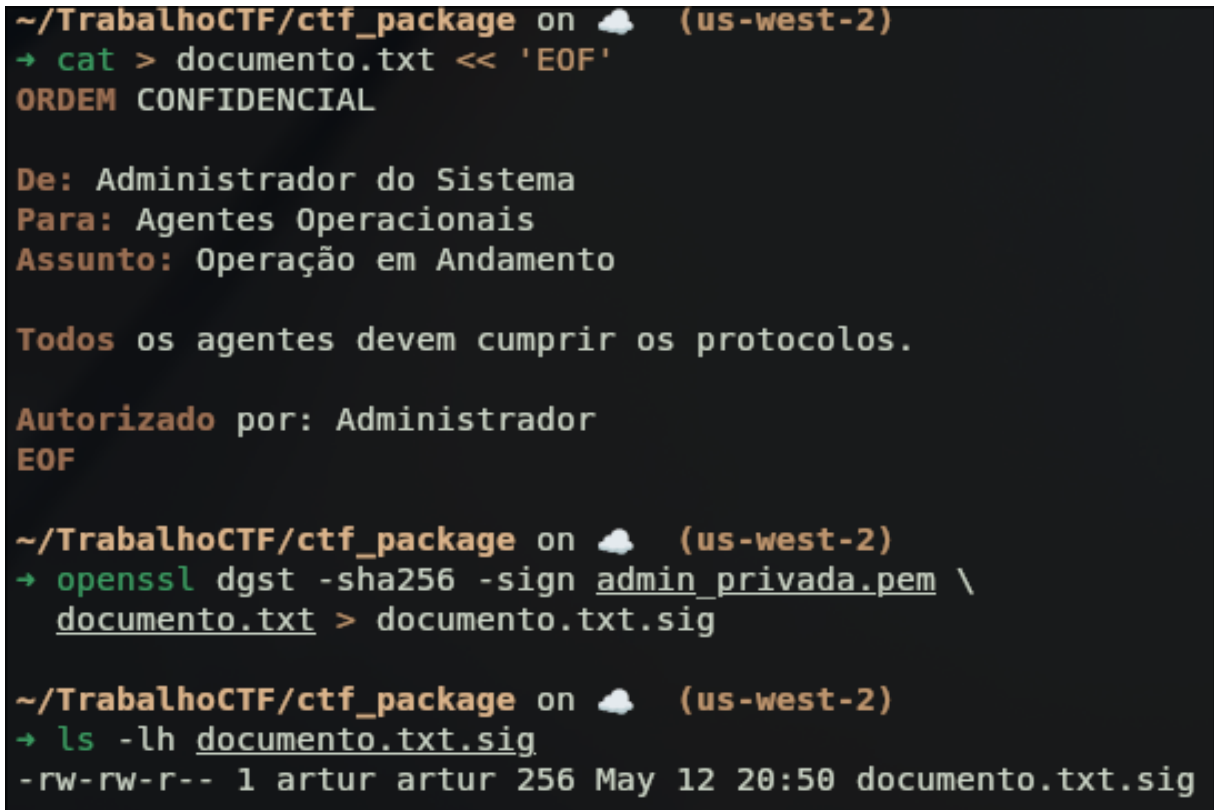
6.2.1 Comandos Utilizados

```
1 # Criar documento de teste
2 cat > documento.txt << 'EOF'
3 ORDEM CONFIDENCIAL
4
5 De: Administrador do Sistema
6 Para: Agentes Operacionais
7 Assunto: Operacao em Andamento
8
9 Todos os agentes devem cumprir os protocolos.
10
11 Autorizado por: Administrador
12 EOF
13
14 # Assinar o documento
15 openssl dgst -sha256 -sign admin_privada.pem \
16     documento.txt > documento.txt.sig
17
18 # Verificar tamanho da assinatura
```

```
19 ls -lh documento.txt.sig
```

Listing 5: Criar e assinar documento

6.2.2 Resultado observado



```
~/TrabalhoCTF/ctf_package on ☁ (us-west-2)
→ cat > documento.txt << 'EOF'
ORDEM CONFIDENCIAL

De: Administrador do Sistema
Para: Agentes Operacionais
Assunto: Operação em Andamento

Todos os agentes devem cumprir os protocolos.

Autorizado por: Administrador
EOF

~/TrabalhoCTF/ctf_package on ☁ (us-west-2)
→ openssl dgst -sha256 -sign admin_privada.pem \
  documento.txt > documento.txt.sig

~/TrabalhoCTF/ctf_package on ☁ (us-west-2)
→ ls -lh documento.txt.sig
-rw-rw-r-- 1 artur artur 256 May 12 20:50 documento.txt.sig
```

Figure 7: Assinatura digital do documento forjado

A execução foi concluída sem erros e gerou o arquivo de assinatura `documento.txt.sig`. O tamanho de **256 bytes** é compatível com assinatura RSA-2048, confirmando que o digest SHA-256 do documento foi assinado com sucesso pela chave privada forjada.

6.3 Mapeamento de Primitivas Criptográficas

As primitivas aplicadas foram função hash SHA-256 e assinatura digital RSA. O propósito arquitetural é garantir integridade e autenticidade do documento. O contorno não exigiu quebra do algoritmo: a assinatura foi produzida corretamente, mas com uma chave privada forjada que o ambiente posteriormente aceita como confiável.

7 Passo 6: Manipulação do Trust Store

7.1 Informações extraídas neste passo

A etapa crítica de exploração arquitetural é alterar o ponto de confiança local do sistema, isto é, o conteúdo do diretório `.trust_store`.

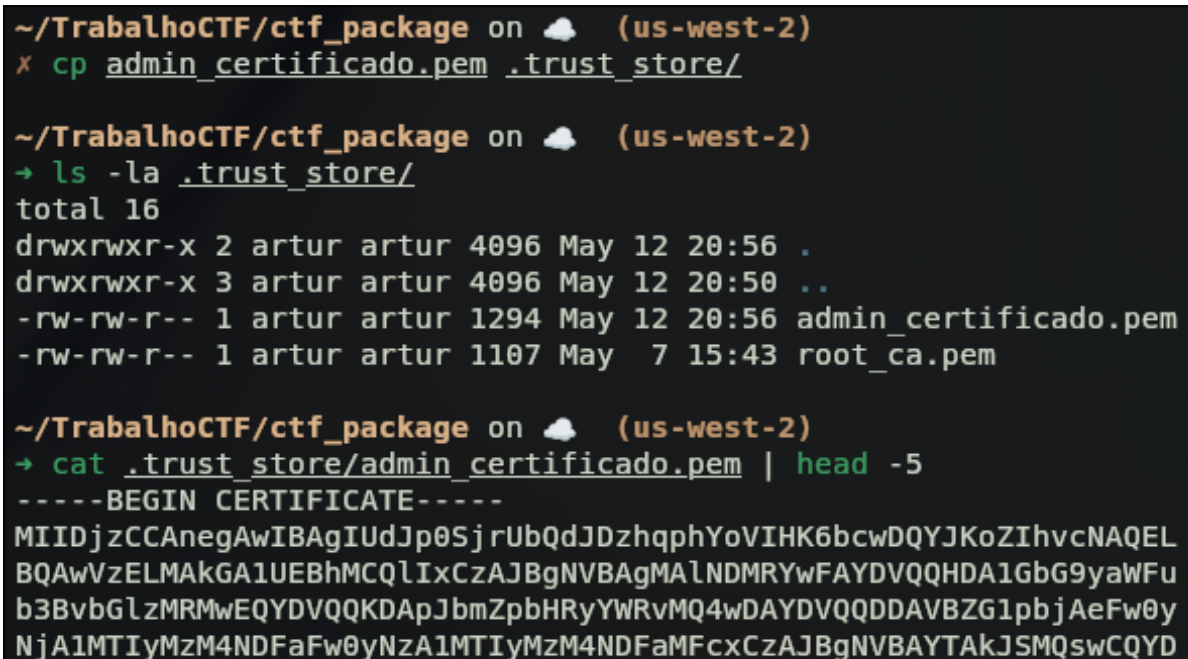
7.2 Metodologia e Vetor de Ataque (Reprodutibilidade)

7.2.1 Comandos Utilizados

```
1 cd ctf_package
2
3 # Copiar certificado para o trust store
4 cp admin_certificado.pem .trust_store/
5
6 # Listar conteúdo do trust store
7 ls -la .trust_store/
8
9 # Verificar se foi adicionado
10 cat .trust_store/admin_certificado.pem | head -5
```

Listing 6: Adicionar certificado ao trust store

7.2.2 Resultado observado



```
~/TrabalhoCTF/ctf_package on 🌩 (us-west-2)
x cp admin_certificado.pem .trust_store/

~/TrabalhoCTF/ctf_package on 🌩 (us-west-2)
→ ls -la .trust_store/
total 16
drwxrwxr-x 2 artur artur 4096 May 12 20:56 .
drwxrwxr-x 3 artur artur 4096 May 12 20:50 ..
-rw-rw-r-- 1 artur artur 1294 May 12 20:56 admin_certificado.pem
-rw-rw-r-- 1 artur artur 1107 May  7 15:43 root_ca.pem

~/TrabalhoCTF/ctf_package on 🌩 (us-west-2)
→ cat .trust_store/admin_certificado.pem | head -5
-----BEGIN CERTIFICATE-----
MIIDjzCCAnegAwIBAgIUdJp0SjrUbQdJDzhqphYoVIHK6bcwDQYJKoZIhvcNAQEL
BQAwVzELMAkGA1UEBhMCQlIx CzAJBgNVBAGMA1NDMRywFAYDVQQQHDA1Gbg9yaWFu
b3BvbGZzMRRwEQYDVQQKDApJbmZpbHRyYWRvMQ4wDAYDVQQDDAVBZG1pbjAeFw0y
NjA1MTIyMzM4NDFaFw0yNzA1MTIyMzM4NDFaMFcx CzAJBgNVBAYTAkJSMSQswCQYD
```

Figure 8: Certificado forjado adicionado ao trust store

O certificado forjado foi copiado com sucesso para o diretório de confiança local por meio do comando `cp admin_certificado.pem .trust_store/`.

Na listagem do trust store, observou-se a presença simultânea do certificado forjado e da autoridade já existente:

```
1 total 16
2 drwxrwxr-x 2 artur artur 4096 May 12 20:56 .
3 drwxrwxr-x 3 artur artur 4096 May 12 20:50 ..
4 -rw-rw-r-- 1 artur artur 1294 May 12 20:56 admin_certificado.pem
5 -rw-rw-r-- 1 artur artur 1107 May 7 15:43 root_ca.pem
```

A inspeção das primeiras linhas confirmou o formato PEM válido do arquivo inserido:

```
1 -----BEGIN CERTIFICATE-----
2 MIIDjzCCAnegAwIBAgIUdJpOSjrUbQdJDzhqphYoVIHK6bcwDQYJKoZIhvcNAQEL
3 BQAwVzELMAkGA1UEBhMCQ1IxCzAJBgNVBAGMA1NDMRYwFAYDVQQHDA1GbG9yaWFu
4 b3BvbG1zMRRMwEQYDVQQKDApJbmZpbHRyYWRvMQ4wDAYDVQQDDAVBZG1pbjAeFw0y
5 NjA1MTIyMzM4NDFaFw0yNzA1MTIyMzM4NDFaMFcxCzAJBgNVBAYTAkJSMQswCQYD
```

Com isso, o trust store foi efetivamente alterado para incluir a identidade forjada como material confiável no processo de validação.

7.3 Mapeamento de Primitivas Criptográficas

Neste passo, o foco foi a primitiva de certificação X.509 no contexto de uma infraestrutura de confiança (PKI simplificada). O propósito arquitetural era restringir validações apenas a emissores confiáveis. O contorno ocorreu ao inserir manualmente o certificado forjado no `.trust_store`, comprometendo a âncora de confiança utilizada pelo verificador.

8 Passo 7: Teste Final - Aprovação do Verificador

8.1 Informações extraídas neste passo

Após comprometer o trust store, resta validar se o fluxo de verificação aceita, na prática, uma assinatura produzida com identidade forjada.

8.2 Metodologia e Vetor de Ataque (Reprodutibilidade)

8.2.1 Comandos Utilizados

```
1 # Gerar HMAC-SHA256 esperado pelo verificador
2 HMAC=$(openssl dgst -sha256 -hmac '1nt3gr1d4d3' documento.txt |
   awk '{print _$NF}')
3
```

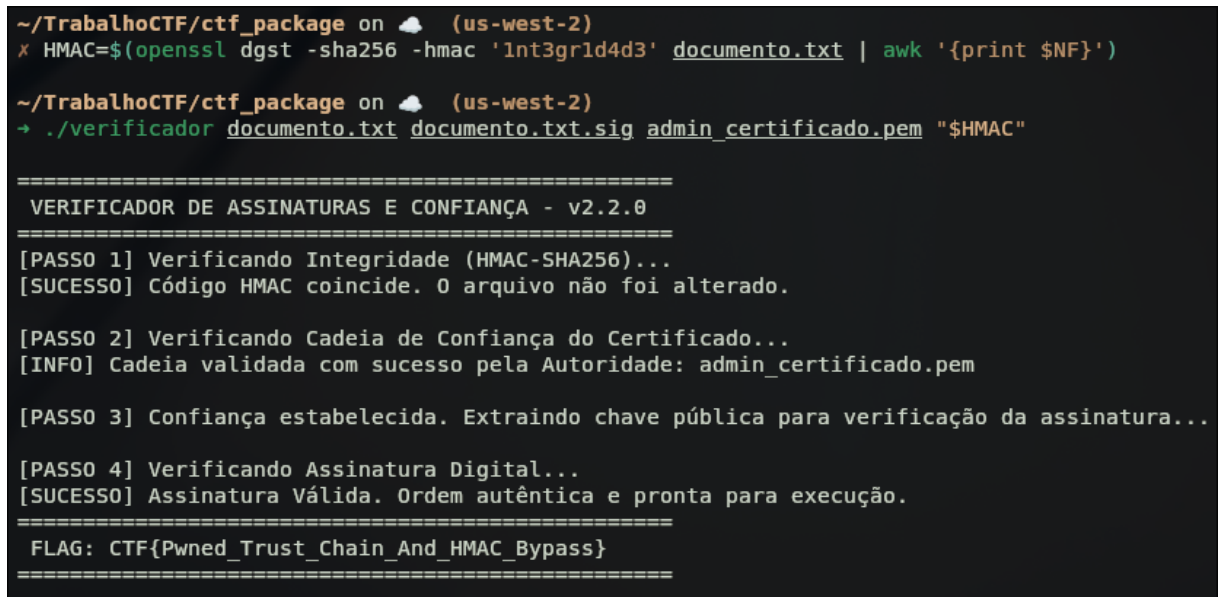
```

4 # Executar o verificador com os 4 argumentos corretos
5 ./verificador documento.txt documento.txt.sig admin_certificado.
  pem "$HMAC"

```

Listing 7: Testar com o verificador

8.2.2 Resultado observado



```

~/TrabalhoCTF/ctf_package on ☁ (us-west-2)
x HMAC=$(openssl dgst -sha256 -hmac '1nt3gr1d4d3' documento.txt | awk '{print $NF}')

~/TrabalhoCTF/ctf_package on ☁ (us-west-2)
→ ./verificador documento.txt documento.txt.sig admin_certificado.pem "$HMAC"

=====
VERIFICADOR DE ASSINATURAS E CONFIANÇA - v2.2.0
=====
[PASSO 1] Verificando Integridade (HMAC-SHA256)...
[SUCESSO] Código HMAC coincide. O arquivo não foi alterado.

[PASSO 2] Verificando Cadeia de Confiança do Certificado...
[INFO] Cadeia validada com sucesso pela Autoridade: admin_certificado.pem

[PASSO 3] Confiança estabelecida. Extraindo chave pública para verificação da assinatura...

[PASSO 4] Verificando Assinatura Digital...
[SUCESSO] Assinatura Válida. Ordem autêntica e pronta para execução.
=====
FLAG: CTF{Pwned_Trust_Chain_And_HMAC_Bypass}
=====

```

Figure 9: **EVIDÊNCIA CRÍTICA:** O verificador aprova o documento forjado

Esse resultado confirma, na prática, que o documento forjado foi aceito pelo mecanismo de verificação após o comprometimento da cadeia de confiança e o fornecimento do código de integridade esperado.

Para fins de reprodutibilidade textual (além da captura de tela), o terminal apresentou, de forma resumida:

```

1 [PASSO 1] Verificando Integridade (HMAC-SHA256)... [SUCESSO]
2 [PASSO 2] Verificando Cadeia de Confiança do Certificado... [
  SUCESSO]
3 [PASSO 4] Verificando Assinatura Digital... [SUCESSO]
4 FLAG: CTF{Pwned_Trust_Chain_And_HMAC_Bypass}

```

8.3 Mapeamento de Primitivas Criptográficas

Na etapa final, o verificador combina assinatura digital (RSA + SHA-256) com validação de cadeia de confiança X.509. O propósito arquitetural desse desenho é aceitar somente documentos assinados por identidades autênticas e confiáveis. O contorno foi efetivo

porque a cadeia de confiança já estava comprometida: a assinatura forjada passa na verificação por ter sido emitida por certificado inserido no trust store do próprio sistema.

9 Recomendações de Mitigação

Com base nas etapas exploradas, as seguintes medidas reduzem significativamente a superfície de ataque observada:

- Remover segredos de integridade hardcoded no cliente e migrar a validação de HMAC para contexto server-side ou para mecanismo com proteção de segredo em hardware.
- Proteger o trust store contra escrita por usuários não privilegiados, incluindo permissões restritivas e validação de cadeia com pinning da CA raiz esperada.
- Exigir validação estrita de certificado (cadeia completa, emissor esperado, período de validade e constraints de uso) antes da verificação da assinatura.
- Implementar trilhas de auditoria e alertas para alterações em `.trust_store` e para tentativas de validação com novos certificados.

10 Conclusão Técnica

O fluxo reproduzido demonstra uma falha arquitetural de confiança e não uma quebra matemática das primitivas criptográficas.

Em sequência, foi possível: recuperar informação-chave no `secret.txt`, decriptar o conteúdo protegido com AES-256-CBC, entender a lógica do `verificador`, produzir identidade e assinatura forjadas com RSA/X.509, comprometer o `.trust_store` e obter aprovação final do artefato forjado.

As evidências visuais permanecem distribuídas ao longo das sessões de Passo 1 a Passo 7, acompanhando cada etapa no momento em que foi executada, como um registro técnico contínuo da operação.

Com o sucesso do ataque, o impacto observado foi:

- Descriptografar dados confidenciais
- Forjar identidades aceitas pelo sistema
- Assinar documentos como autoridades do sistema
- Infiltrar-se completamente na infraestrutura

Conclusão: O sistema foi completamente comprometido.